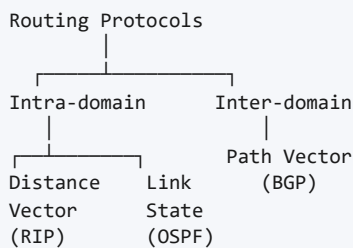


Routing Protocols

1. Autonomous Systems & Routing Classification

Autonomous System (AS) — a group of networks and routers under the authority of a **single administration** (e.g., one company's or one ISP's entire network).

Routing inside an AS → intra-domain (interior) routing
Routing between ASes → inter-domain (exterior) routing



Each protocol answers "how do I find the best path?" using a fundamentally different **algorithm** and a different **scope**.

2. Distance Vector Routing (the algorithm behind RIP)

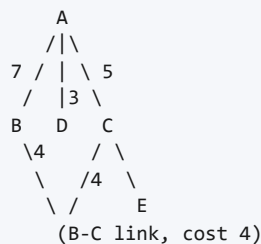
The core idea

Each router only knows the cost to reach directly-connected networks initially. Routers periodically share their **entire routing table** with their **immediate neighbors only** — never any information about the whole network's topology, and never with routers further away.

Three phases:

1. **Initialization** — each router builds a table containing only the networks it's **directly connected** to, with cost 1 (or whatever the direct link's cost is) and no next-hop needed.
2. **Sharing** — periodically (and whenever something changes), each router sends its **complete current table** to its immediate neighbors.
3. **Updating** — on receiving a neighbor's table, a router checks: *"can I reach some destination more cheaply by routing through this neighbor than via what I currently know?"* If yes, it updates its own table (new cost = neighbor's advertised cost + cost to reach that neighbor; next hop = that neighbor).

Worked example



Edges: A–B (7), A–C (5), A–D (3), B–C (4), C–E (4). (D and E are leaf nodes, reachable only through A and C respectively.)

Initialization (direct neighbors only):

```
| Router | Knows |
|---|---|
| A | B=7, C=5, D=3 |
| B | A=7, C=4 |
| C | A=5, B=4, E=4 |
| D | A=3 |
| E | C=4 |
```

After sharing + updating (converged tables):

From \ To	A	B	C	D	E
A	—	7 (direct)	5 (direct)	3 (direct)	9 (via C)
B	7 (direct)	—	4 (direct)	10 (via A)	8 (via C)
C	5 (direct)	4 (direct)	—	8 (via A)	4 (direct)
D	3 (direct)	10 (via A)	8 (via A)	—	12 (via A→C)
E	9 (via C)	8 (via C)	4 (direct)	12 (via C→A)	—

How B's entry for D got computed: B initially has no idea D exists. A shares its table (which includes **D=3**) with B. B computes: cost to reach A (7) + A's advertised cost to D (3) = **10**, via next-hop A. This exact "neighbor's advertised cost + my cost to that neighbor" computation is the entire distance-vector algorithm, applied repeatedly until every router's table stabilizes (converges).

Interview soundbite: "Distance vector routing is router-to-router gossip with zero map-awareness — each router just tells its neighbors 'here's what I currently think the cost to reach everywhere is,' and every router blindly trusts and builds on what its neighbors report, without ever seeing the network's actual topology."

3. RIP (Routing Information Protocol)

RIP is the classic **intra-domain, distance-vector** protocol — it's literally the distance vector algorithm above, standardized as a real protocol.

RIP Messages

- **Request** — sent by a router that just came up, or has some entries that timed out, asking for routing info (can ask for specific entries or everything).
- **Response** — either **solicited** (a direct reply to a request) or **unsolicited** (sent proactively every 25–30 seconds, or immediately whenever the table changes).

RIP Timers

Timer	Duration	Purpose
Periodic timer	25–30 sec	Controls how often regular (unsolicited) update messages are advertised
Expiration timer	180 sec	Governs the validity of a specific route; if no update refreshes it in time, the route is considered expired and its hop count is set to 16 (= infinity in RIP)
Garbage collection timer	120 sec	An invalid (expired) route is not immediately deleted — it's kept (advertised as unreachable) until this timer also expires, giving neighbors time to learn it's gone before it disappears entirely

Worked example: a routing table has 20 entries. It hasn't received any update for 5 of those routes in 200 seconds. How many timers are currently running?

```
Periodic timer:      1  (there's only ever one, shared across the whole table)
Expiration timers:  20 - 5 = 15  (one per still-valid route)
Garbage collection timers:  5  (one per the 5 routes that have now expired, at 200s > 180s expiration)

Total = 1 + 15 + 5 = 21 timers
```

The Count-to-Infinity Problem

Distance vector's biggest structural weakness: when a link/router fails, **bad news travels slowly** (recall each router only trusts and shares full-table gossip, not verified topology), and routers can end up in a loop feeding each other incorrect, ever-increasing costs before the failure is finally recognized.

A — B — C (originally: B's cost to C = 1, A's cost to C via B = 2)

C fails. B notices directly (its own link to C is down), sets its cost to C = ∞ (16 in RIP).

But then A sends B its OLD table, which still says "A can reach C with cost 2."

B (not knowing A's route to C actually depended on B itself) thinks:

"Oh, I can reach C via A for cost $2+1=3$!" — and updates its table to 3.

B advertises this "3" back to A.

A now thinks: "B says cost 3, so via B, my cost to C = $3+1 = 4$ " — updates to 4.

... this ping-pongs upward, slowly climbing toward infinity (16 in RIP), one increment at a time, taking many rounds to finally converge.

Fixes:

- **Split Horizon** — a router **never** advertises a route back to the same neighbor it originally learned that route from (since that neighbor's own path to the destination almost certainly depends on going back through the router being told — advertising it back is pointless and dangerous).

- **Split Horizon with Poison Reverse** — a stronger variant: instead of just *omitting* that route when advertising back to the source neighbor, actively advertise it back with a cost of **infinity** — an explicit "don't route this destination through me" signal, which propagates the bad news faster and more unambiguously than silence alone would.

Interview soundbite: "Count-to-infinity happens because distance vector routers trust their neighbors' summaries without knowing *why* those numbers are what they are — so stale information can get fed right back into the system and mistaken for a new, valid route. Split horizon (and its stronger poison-reverse variant) breaks this specific failure mode by never telling a neighbor 'you can reach X through me' when that neighbor is literally the reason the router believes it can reach X in the first place."

4. Link State Routing (the algorithm behind OSPF)

A fundamentally different philosophy from distance vector:

Instead of trusting neighbors' summarized tables, **every router builds a complete, identical map of the entire network's topology**, and independently computes shortest paths from that map.

How it works, at a high level:

1. Each router discovers its immediate neighbors and the **cost/state of each link** to them.
2. Each router **floods** this link-state information to **every other router** in the network (not just immediate neighbors) — so eventually, every router has received everyone else's link-state advertisements.
3. Every router now has an **identical, complete topology map** of the network.

4. Each router independently runs **Dijkstra's shortest-path algorithm** on this map, from itself as the source, to compute optimal routes to every destination.

OSPF (Open Shortest Path First) is the standard link-state intra-domain protocol. It also introduces **areas** — dividing a large AS into smaller regions, so link-state flooding (which is more bandwidth/CPU-intensive than distance-vector's simple neighbor gossip) stays contained within a manageable area rather than flooding an entire huge network.

Distance Vector vs Link State

	Distance Vector (RIP)	Link State (OSPF)
What's shared	Entire routing table, to immediate neighbors only	Link-state info, flooded to everyone
Topology knowledge	None — only "cost to X," no map	Complete, identical map at every router
Algorithm	Bellman-Ford-style iterative updates	Dijkstra's shortest path, run locally
Convergence	Slow (count-to-infinity risk)	Fast, more robust
Overhead	Low per-message, but many rounds to converge	Higher per-message (full flooding), but converges faster

Interview soundbite: "Distance vector is 'trust your neighbor's summary.' Link state is 'get the raw facts from everyone, and compute your own answer.' Link state converges faster and avoids count-to-infinity entirely, because a router that has the actual topology map can't be fooled by stale secondhand information the way a distance-vector router can — but it costs more to flood that much information to the entire network."

5. Path Vector Routing (the algorithm behind BGP)

Used specifically for **inter-domain** routing — routing *between* different Autonomous Systems, which is a fundamentally different problem than routing *within* one.

Why distance vector and link state don't fit here:

- Trusting neighbor AS's cost claims blindly (like distance vector) is dangerous between organizations that don't necessarily trust each other's numbers.
- Flooding full topology to every router across every AS on the entire Internet (like link state) would be a catastrophic amount of information for a network the size of the Internet.
- Between ASes, the deciding factor usually isn't even "shortest cost" — it's **policy** (business agreements, peering relationships, "never route through a competitor's network," etc.).

Path Vector's approach: instead of advertising just a cost, each route advertisement includes the **entire list (path) of ASes** it would traverse to reach the destination.

AS advertises: "I can reach network X via the path: AS3 → AS7 → AS12"

- A receiving AS can then apply its own **policy** to decide whether to accept/prefer this route — e.g., refuse any path that passes through a specific competitor AS, regardless of whether it's technically the shortest.
- **Loop prevention is immediate and simple:** if an AS sees its own AS number already present in an advertised path, it just rejects that route outright — no count-to-infinity-style iterative convergence needed at all.

BGP (Border Gateway Protocol) is the (essentially only) real-world inter-domain routing protocol — literally what holds the entire global Internet's inter-AS routing together.

- **eBGP** — BGP sessions between routers in *different* ASes.

- **iBGP** — BGP sessions between routers *within* the same AS, used to distribute externally-learned BGP routes internally.

Interview soundbite: "BGP isn't optimizing for shortest path at all — it's optimizing for policy compliance between mutually distrustful, independently-operated networks. The AS-path itself doubles as both the routing metric input and a trivial, built-in loop-detection mechanism — if your own AS number is already in the path, you know instantly that accepting the route would create a loop."

Interview Questions With Answers

Q1. In distance vector routing, what exact information does a router share with its neighbors, and how often?

Answer: A router shares its **entire current routing table** — its believed cost to reach every known destination — with only its **immediate neighbors**, not the whole network. This is shared periodically (every 25–30 seconds in RIP) and also immediately whenever the table changes, so neighbors can react to updates promptly rather than waiting for the next scheduled cycle.

Q2. Walk through how a distance-vector router updates its table when it receives a neighbor's advertisement.

Answer: For every destination the neighbor advertises a cost for, the receiving router computes a candidate cost as `(cost to reach that neighbor) + (neighbor's advertised cost to the destination)`. If this candidate cost is lower than what the router currently believes for that destination, it updates its table entry to this new, lower cost, and sets the next-hop for that destination to be that neighbor.

Q3. What causes the count-to-infinity problem, and what makes it specifically a distance-vector issue rather than a link-state one?

Answer: It's caused by routers trusting secondhand, already-stale summarized information from neighbors without knowing *why* that information is what it is — when a link fails, a neighbor can unknowingly feed back an old, no-longer-valid route that actually depended on the very router now asking about it, causing both routers to iteratively inflate the cost estimate step by step rather than immediately recognizing the destination is unreachable. This is specific to distance vector because it only ever exchanges summarized costs, never the underlying topology; link-state routers each hold the complete, actual topology map, so they can directly determine reachability is broken rather than inferring it indirectly through a chain of secondhand cost updates.

Q4. How does split horizon prevent (or reduce) the count-to-infinity problem, and how does poison reverse improve on plain split horizon?

Answer: Split horizon prevents a router from advertising a route back to the exact neighbor it originally learned that route from — since that neighbor's own path to the destination is almost certainly routed back through the very router being told, re-advertising it back is both pointless and specifically what causes the bad feedback loop. Poison reverse strengthens this by not just omitting the route in that direction, but actively advertising it with a cost of infinity — an explicit, unambiguous "you cannot reach this destination through me" signal that propagates the failure information faster and more reliably than silent omission alone.

Q5. Explain, step by step, how link-state routing avoids needing to trust any single neighbor's summarized claims.

Answer: Every router independently discovers only its own directly-connected links and their costs — this is the only "local" information it ever originates. That local information is then flooded, essentially unmodified, to every other router in the network, rather than being combined/summarized along the way. Because every router receives the same complete set of raw link-state facts from every other router, each one can independently reconstruct an identical, accurate topology map and compute its own shortest paths (via Dijkstra) directly from that map — there's no step where one router has to trust another router's *computed conclusions* about distances, only trust the raw, unprocessed facts about what that router's own direct links look like.

Q6. Why does OSPF divide a large network into areas, and what would happen without this division?

Answer: Link-state routing requires flooding link-state information to every router in the routed domain, and having every router in a very large network maintain and recompute Dijkstra over an enormous, constantly-changing topology map imposes significant bandwidth and CPU overhead. Dividing the network into areas contains this flooding within each smaller area, so a change in one area doesn't force every router across the entire large network to recompute its full topology map — without this division, link-state routing's overhead would scale poorly as the network grows very large, undermining the fast-convergence benefit that makes link-state attractive in the first place.

Q7. Why is BGP's approach (path vector) necessary between Autonomous Systems, rather than just running RIP or OSPF across the whole Internet?

Answer: Distance vector (RIP) would require ASes to blindly trust each other's cost claims, which is unacceptable between independently-operated, mutually distrustful organizations, and it inherits the count-to-infinity risk at Internet scale. Link state (OSPF) would require flooding complete topology information to every router across the entire Internet, an infeasible amount of information and churn for a network of that size and rate of change. Beyond the scaling issue, inter-AS routing decisions are fundamentally driven by **business policy** (peering agreements, refusing to route through certain networks) rather than pure shortest-cost — something neither RIP's cost-only nor OSPF's shortest-path-only model is designed to express; BGP's path vector approach carries the full AS-path specifically so that policy decisions can be applied per-route.

Q8. How does path vector routing detect and prevent loops, and why is this simpler than distance vector's approach?

Answer: Every route advertisement in path vector routing carries the complete list of ASes it has already traversed. A router receiving an advertisement simply checks whether its own AS number already appears somewhere in that path — if so, accepting the route would create a loop, so it's rejected outright, immediately and with certainty. This is simpler than distance vector's approach (which relies on techniques like split horizon/poison reverse and can still suffer slow convergence via count-to-infinity in some topologies) because the loop information is carried explicitly and completely in the advertisement itself, requiring no iterative convergence or inference — a single direct check settles the matter.

Q9. Scenario: Two internal routers within the same company's network need to exchange routing information, and the company also peers with three external ISPs at its network edge. Which routing protocol(s) would typically be used where, and why?

Answer: Within the company's own network (a single AS), an intra-domain protocol like OSPF (link-state) would typically handle routing between internal routers — the company fully controls and trusts its own internal topology, so flooding complete link-state information internally is both feasible and gives fast, robust convergence. At the network edge, where the company's routers peer with three external ISPs (each its own independent AS), BGP would be used — specifically eBGP sessions with each ISP's border routers, since inter-AS routing needs policy-based path selection and mutual-distrust-safe loop prevention, not a shared trusted topology map. Internally, iBGP would then typically be used to distribute the externally-learned BGP routes (received via eBGP from the ISPs) to the rest of the company's internal routers, so that internal routing decisions can correctly account for which internal router has the best path out to each external destination.